STM32F103RB ADC-Based Three-Stage LED Control

STM32F103RB Microcontroller Hardware Abstraction Layer (HAL)

Prepared By: Ahsan Basharat | Status: Intern | Submitted To: Sir Yasir

Objectives

- To understand the basic concept and working principle of ADC (Analog-to-Digital Conversion) for reading analog signals.
- To configure the ADC1 peripheral of the STM32F103RB using the STM32 HAL Library.
- To interface a potentiometer with the ADC and convert its analog voltage into a digital ADC value.
- To divide the ADC range into three levels and control three LEDs according to the potentiometer position.
- To gain hands-on experience in developing, programming, debugging, and verifying ADC and GPIO functionality using STM32CubeIDE and STM32F103RB.
- To implement a structured HAL-based and MISRA-C-style software design using separate .h and .c driver files.

Introduction

This project demonstrates ADC-based LED control using the STM32F103RB. A potentiometer is connected to ADC1 Channel 0 (PA0), and its analog voltage is converted into a 12-bit digital value. Based on the ADC level, three LEDs connected to PA5, PA6, and PA7 are controlled. The project uses the STM32 HAL Library and follows MISRA-C coding practices for reliable and maintainable code.

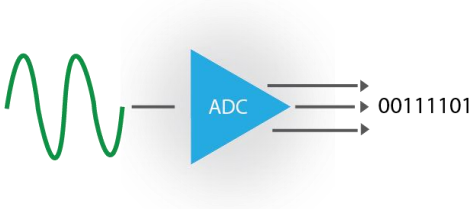
ADC

ADC (Analog-to-Digital Converter) is a hardware peripheral that converts an analog voltage signal into a digital value that a microcontroller can understand and process.

For example, a potentiometer produces a variable analog voltage from 0 V to 3.3 V. The STM32 ADC converts this voltage into a digital number.

For the STM32F103RB, the ADC is 12-bit, so:

- 0 V → 0
- 1.65 V → ~2048
- 3.3 V → 4095



The ADC value can then be used by your program to control things such as LED brightness, motor speed, temperature readings, or sensors.

Required Components

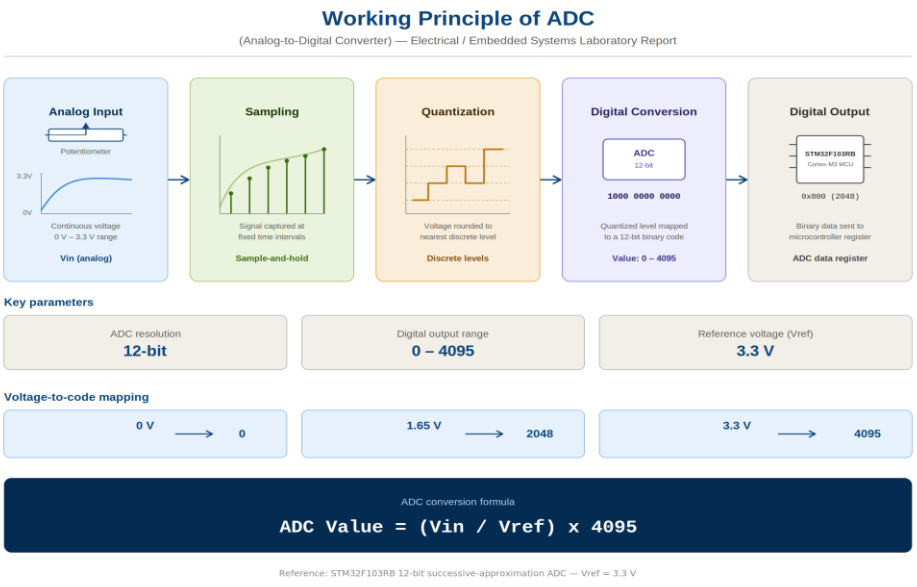
#	Component / Tool	Quantity	Description / Specifications
1	STM32F103RB	1	ARM Cortex-M3 (32-bit)
2	USB Cable / ST-Link V2	1	Hardware programmer and debugger
3	Breadboard	1	Standard solderless prototyping breadboard
4	Light Emitting Diode (LEDS)	3	5mm Red/Green LED indicator
5	Jumper Wires	Set	Male-to-Male and Male-to-Female wires
6	STM32CubeIDE/MX	Software	Integrated Development Environment for C/C++
7	Potentiometer	1	To Control LEDS Brightness

Working Principle of ADC

An ADC (Analog-to-Digital Converter) converts a continuous analog voltage into a digital value that a microcontroller can understand.

The working process is:

- **Analog Input**: An analog voltage, such as the output of a potentiometer, is applied to the ADC input.
- **Sampling:** The ADC samples the input voltage at a specific time.
- **Quantization:** The sampled voltage is compared with the reference voltage and divided into discrete levels.
- **Digital Conversion:** The ADC assigns a digital value corresponding to the input voltage.

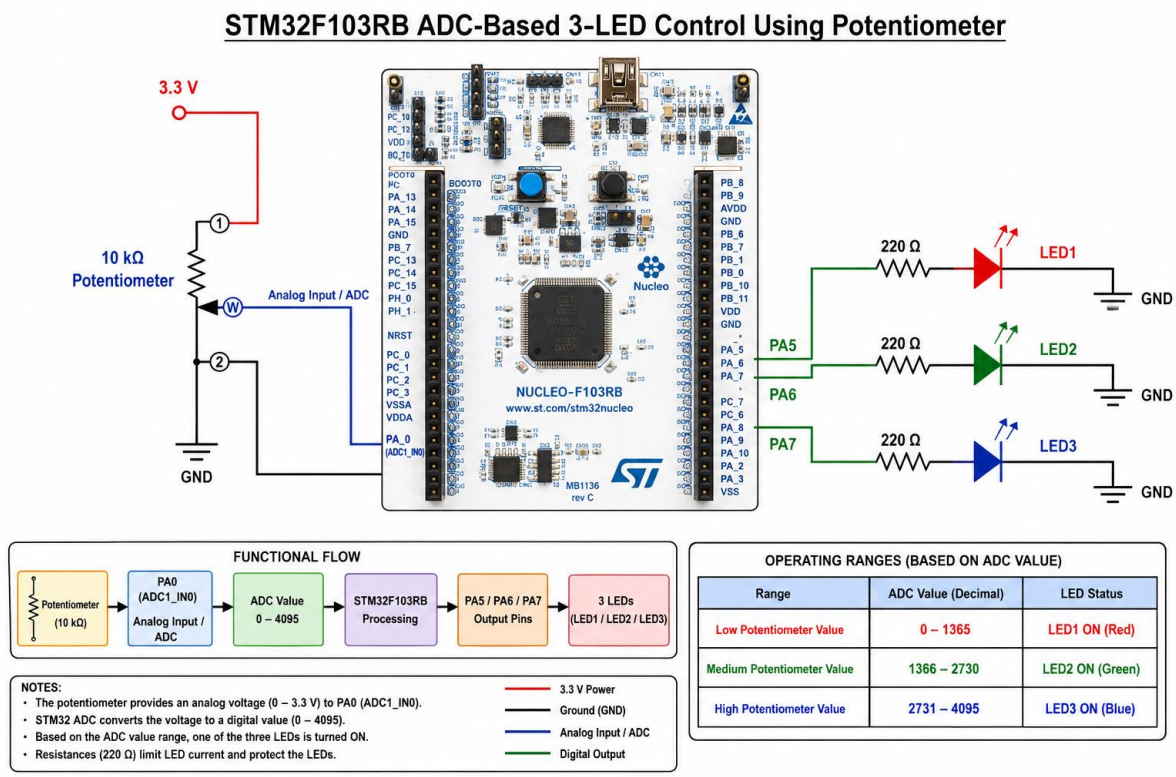


- **Digital Output:** The microcontroller receives and processes this digital value.

For the STM32F103RB, the ADC has a 12-bit resolution, providing 4096 levels (0–4095).

Circuit Diagram of ADC-Based 3-LED Control Using Potentiometer

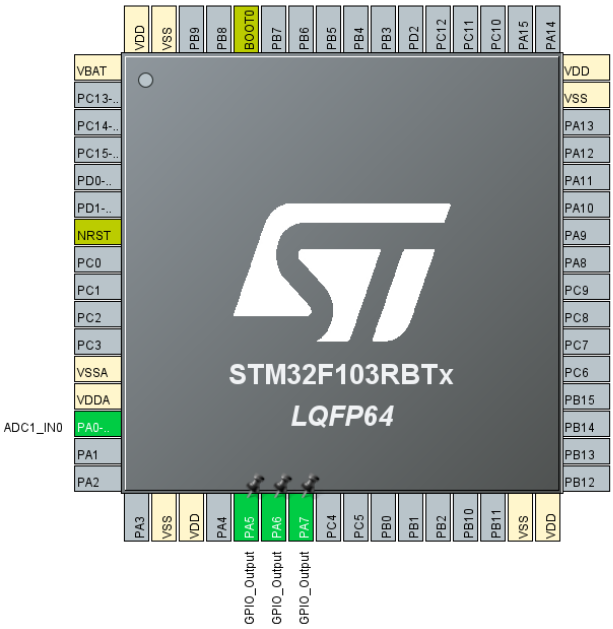
The circuit demonstrates how the STM32 Nucleo-F103RB can use its ADC peripheral to read an analog signal and control multiple LEDs . A 10 kΩ potentiometer is connected between the 3.3 V supply and GND, providing a variable voltage at its middle terminal. The potentiometer wiper is connected to PA0 (ADC1_IN0), where the STM32 reads the changing analog voltage The 12-bit ADC converts the input voltage into a digital value ranging from 0 to 4095, which is processed by the microcontroller. The three LEDs are connected to PA5, PA6, and PA7, with each LED connected through a 220 Ω current-limiting resistor. When the potentiometer is rotated, the ADC value changes and the STM32 activates the corresponding LED according to the defined voltage ranges. For an ADC value of 0–1365, LED1 connected to PA5 is turned ON, indicating a low input level. For an ADC value of 1366–2730, LED2 connected to PA6 is turned ON, representing a medium input level. For an ADC value of 2731–4095, LED3 connected to PA7 is turned ON, representing a high input level. This circuit provides a simple practical demonstration of analog-to-digital conversion, GPIO control, and sensor-based decision making using the STM32 microcontroller.



Procedure

1. Launch **STM32CubeIDE** and create a new project by selecting the **STM32F103RB** microcontroller.
2. Open the `.ioc` configuration file and enable the required peripheral:
   - **ADC1 → Single Conversion Mode**
3. Configure **ADC1** with the following settings:
   - **Resolution:** 12 Bits
   - **Data Alignment:** Right
   - **Scan Conversion Mode:** Disable
   - **Continuous Conversion Mode:** Enable
   - **External Trigger:** Software Start
   - **Channel:** ADC1_IN0 (PA0)
4. Configure the **PA0 GPIO pin** for analog input:
   - **PA0 → ADC1_IN0 (Analog Input)**
   - Connect the middle pin of the potentiometer to **PA0**.
   - Connect the potentiometer's other two pins to **3.3V** and **GND**.
5. Configure the GPIO pins for LED indication:
   - **PA5 → LED1**
   - **PA6 → LED2**
   - **PA7 → LED3**
   - Configure all LED pins as **GPIO Output Push-Pull**.
6. Save the `.ioc` file and allow **STM32CubeIDE** to generate the initialization code.
7. In the user code, initialize and start the ADC:
   - Start **ADC1** using the HAL ADC functions.
   - Read the converted ADC value from **PA0**.
   - The ADC provides a **12-bit digital value from 0 to 4095**.
8. Write the program to:
   - Read the analog voltage from the potentiometer.
   - Compare the ADC value with predefined thresholds.
   - Control the three LEDs according to the potentiometer position:
     - **ADC < 1365:** LED1 ON, LED2 and LED3 OFF.
     - **ADC = 1365–2730:** LED2 ON, LED1 and LED3 OFF.
     - **ADC > 2730:** LED3 ON, LED1 and LED2 OFF.
9. Build the project and program the **STM32F103RB** using the **ST-LINK/V2** debugger.
10. Adjust the potentiometer connected to **PA0** and observe the LEDs. As the analog input voltage changes from **0V to 3.3V**, the ADC value changes from approximately **0 to 4095**, causing the LEDs to operate according to the defined ADC ranges.

Code:




```c
/* ADC Handle Structure */
typedef struct
{
    ADC_HandleTypeDef hadc;
} ADC_HAL;

/*****************************************************************************
 * Function    : ADC_Init
 *
 * Description : Initializes ADC1 peripheral and configures PA0 as
 *               analog input.
 *
 * Parameter   : adc - Pointer to ADC object.
 *
 * Return      : None
 *****************************************************************************/
void ADC_Init(ADC_HAL * const adc);

/*****************************************************************************
 * Function    : ADC_Read
 *
 * Description : Starts ADC conversion and returns converted value.
 *
 * Parameter   : adc - Pointer to ADC object.
 *
 * Return      : 12-bit ADC conversion result (0 - 4095).
 *****************************************************************************/
uint16_t ADC_Read(ADC_HAL * const adc);

#endif /* ADC_HAL_H */
```

ADC_HAL Structure: Contains an ADC_HandleTypeDef for managing the ADC peripheral.

ADC_Init(): Initializes ADC1 and configures PA0 as the analog input.

ADC_Read(): Starts an ADC conversion and returns the 12-bit digital value (0–4095).

```c
void ADC_Init(ADC_HAL * const adc)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    ADC_ChannelConfTypeDef sConfig = {0};

    if (adc != NULL)
    {
        /* Enable GPIOA peripheral clock */
        __HAL_RCC_GPIOA_CLK_ENABLE();

        /* Enable ADC1 peripheral clock */
        __HAL_RCC_ADC1_CLK_ENABLE();

        /* Configure PA0 as Analog Input */
        GPIO_InitStruct.Pin  = GPIO_PIN_0;
        GPIO_InitStruct.Mode = GPIO_MODE_ANALOG;

        HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

        /* Configure ADC1 parameters */
        adc->hadc.Instance = ADC1;
        adc->hadc.Init.ScanConvMode = ADC_SCAN_DISABLE;
        adc->hadc.Init.ContinuousConvMode = DISABLE;
        adc->hadc.Init.DiscontinuousConvMode = DISABLE;
        adc->hadc.Init.ExternalTrigConv = ADC_SOFTWARE_START;
        adc->hadc.Init.DataAlign = ADC_DATAALIGN_RIGHT;
        adc->hadc.Init.NbrOfConversion = 1U;

        (void)HAL_ADC_Init(&adc->hadc);

        /* Configure ADC Regular Channel */
        sConfig.Channel = ADC_CHANNEL_0;
        sConfig.Rank = ADC_REGULAR_RANK_1;
        sConfig.SamplingTime = ADC_SAMPLETIME_55CYCLES_5;

        (void)HAL_ADC_ConfigChannel(&adc->hadc, &sConfig);

        /* Perform ADC Calibration */
        (void)HAL_ADCEx_Calibration_Start(&adc->hadc);
    }
}
```

- Checks that the ADC object pointer is not NULL.
- Enables the GPIOA and ADC1 peripheral clocks.
- Configures PA0 as an analog input.
- Sets up ADC1 for single-channel, single-conversion operation.
- Configures ADC Channel 0 with 55.5-cycle sampling time.
- Initializes and calibrates the ADC using STM32 HAL functions.

```c
uint16_t ADC_Read(ADC_HAL * const adc)
{
    uint16_t value = 0U;

    if (adc != NULL)
    {
        /* Start ADC Conversion */
        (void)HAL_ADC_Start(&adc->hadc);

        /* Wait until conversion is complete */
        (void)HAL_ADC_PollForConversion(&adc->hadc, HAL_MAX_DELAY);

        /* Read converted ADC value */
        value = (uint16_t)HAL_ADC_GetValue(&adc->hadc);

        /* Stop ADC Conversion */
        (void)HAL_ADC_Stop(&adc->hadc);
    }

    return value;
}
```

- Initializes the ADC value to 0.
- Checks whether the ADC pointer is not NULL.
- Starts the ADC conversion using HAL_ADC_Start().
- Waits for the conversion to complete using HAL_ADC_PollForConversion().
- Reads the 12-bit ADC value (0–4095) using HAL_ADC_GetValue().
- Stops the ADC conversion and returns the converted value.

Main.c

```
/* ADC Threshold Values */
#define ADC_LOW_THRESHOLD       (1365U)
#define ADC_HIGH_THRESHOLD      (2730U)

/* Delay between ADC Samples (ms) */
#define ADC_SAMPLE_DELAY_MS     (10U)
```

- ADC_LOW_THRESHOLD (1365U) defines the lower ADC threshold.
- ADC_HIGH_THRESHOLD (2730U) defines the upper ADC threshold.
- Since the ADC range is 0~4095, these thresholds divide the input into three levels.
- ADC_SAMPLE_DELAY_MS (10U) sets a 10 ms delay between ADC readings.
- These values are used to control LED stages based on the potentiometer position.

```
/* LED connected to GPIOA Pin 5 */
GPIO_HAL LED =
{
    GPIOA,
    GPIO_PIN_5
};

/* LED connected to GPIOA Pin 6 */
GPIO_HAL LED1 =
{
    GPIOA,
    GPIO_PIN_6
};

/* LED connected to GPIOA Pin 7 */
GPIO_HAL LED2 =
{
    GPIOA,
    GPIO_PIN_7
};
```

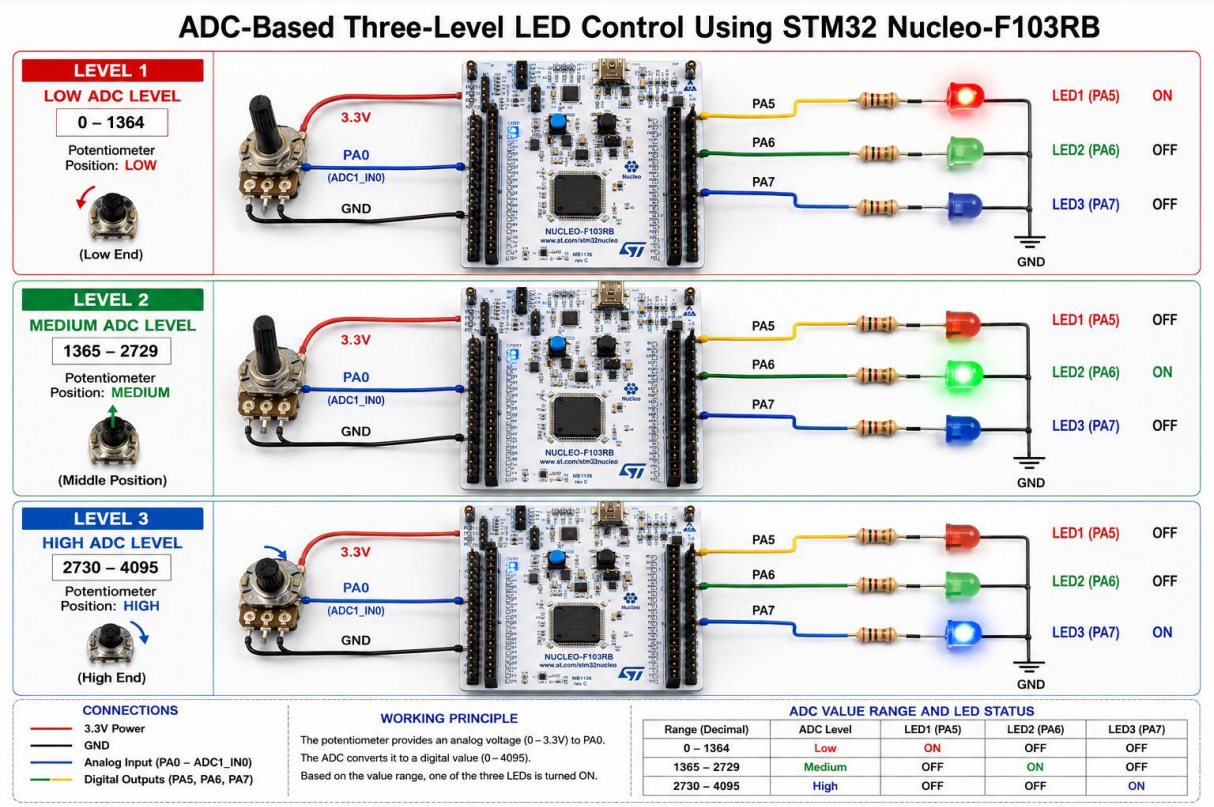
- LED: Connected to PA5.
- LED1: Connected to PA6.
- LED2: Connected to PA7.
- ADC1_Driver is the global ADC HAL object used to manage ADC1.
- ADC_Value stores the 12-bit ADC conversion result.
- ADC_Value is initialized to 0 and declared static for file-level use.

```
/*****************************************************************
 *                        Global ADC Object
 *****************************************************************/

ADC_HAL ADC1_Driver;

/*****************************************************************
 *                        Global Variables
 *****************************************************************/

/* Stores ADC Conversion Result */
static uint16_t ADC_Value = 0U;
```

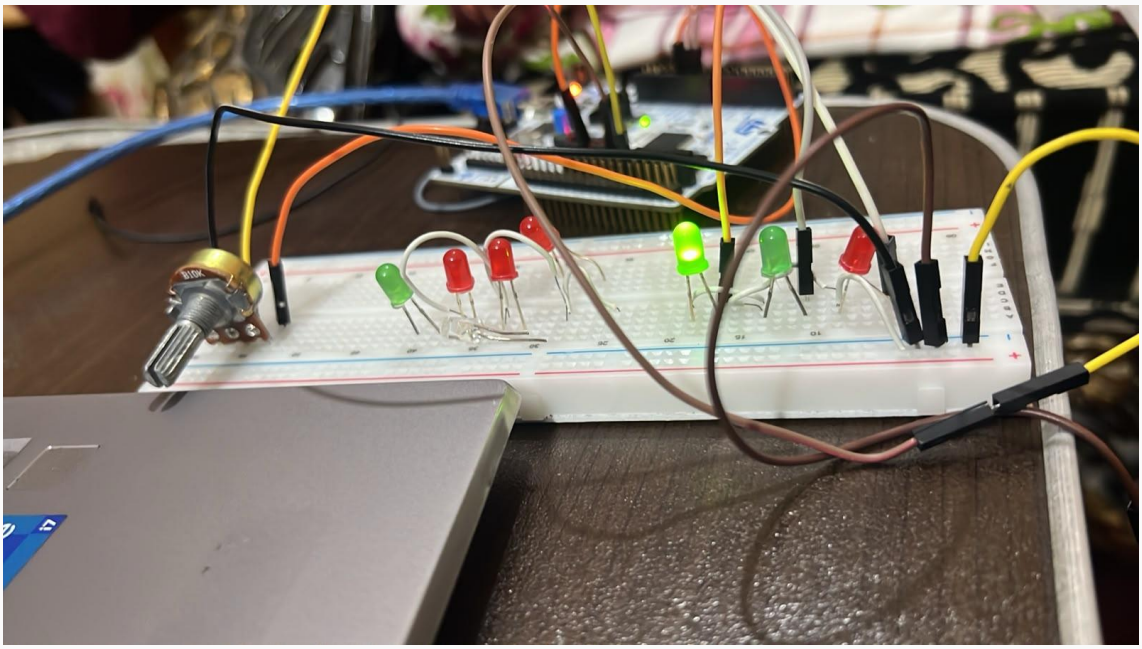
```c
    LED_Init(&LED);
    LED_Init(&LED1);
    LED_Init(&LED2);

    /* Initialize ADC Peripheral */
    ADC_Init(&ADC1_Driver);

    /* Infinite Loop */
    while (1)
    {
        /* Read Potentiometer Value */
        ADC_Value = ADC_Read(&ADC1_Driver);

        /* Low ADC Range */
        if (ADC_Value < ADC_LOW_THRESHOLD)
        {
            LED_On(&LED);
            LED_Off(&LED1);
            LED_Off(&LED2);
        }
        /* Medium ADC Range */
        else if (ADC_Value < ADC_HIGH_THRESHOLD)
        {
            LED_Off(&LED);
            LED_On(&LED1);
            LED_Off(&LED2);
        }
        /* High ADC Range */
        else
        {
            LED_Off(&LED);
            LED_Off(&LED1);
            LED_On(&LED2);
        }

        /* Wait before next ADC conversion */
        HAL_Delay(ADC_SAMPLE_DELAY_MS);
    }
```

- Initializes all three LEDs using LED_Init().
- Initializes the ADC using ADC_Init().
- Continuously reads the potentiometer value through ADC_Read().
- Low range (0–1364): PA5 LED turns ON.
- Medium range (1365–2729): PA6 LED turns ON.
- High range (2730–4095): PA7 LED turns ON.
- Adds a 10 ms delay between ADC readings for stable operation.

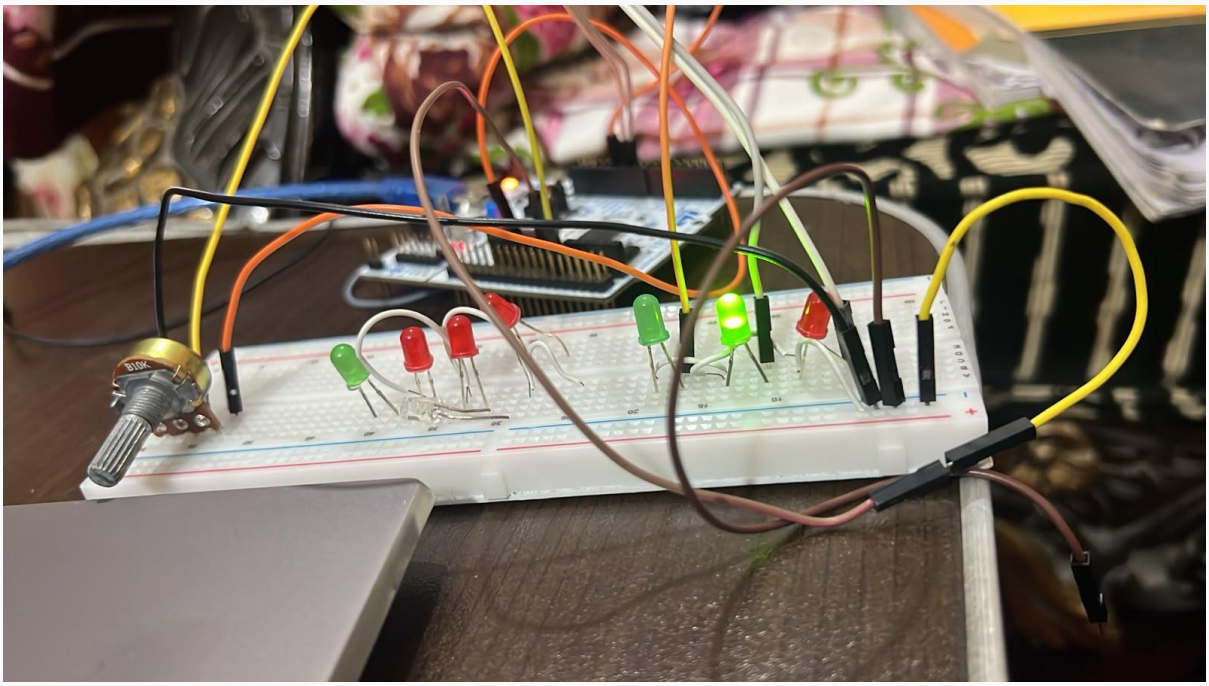
Program Output



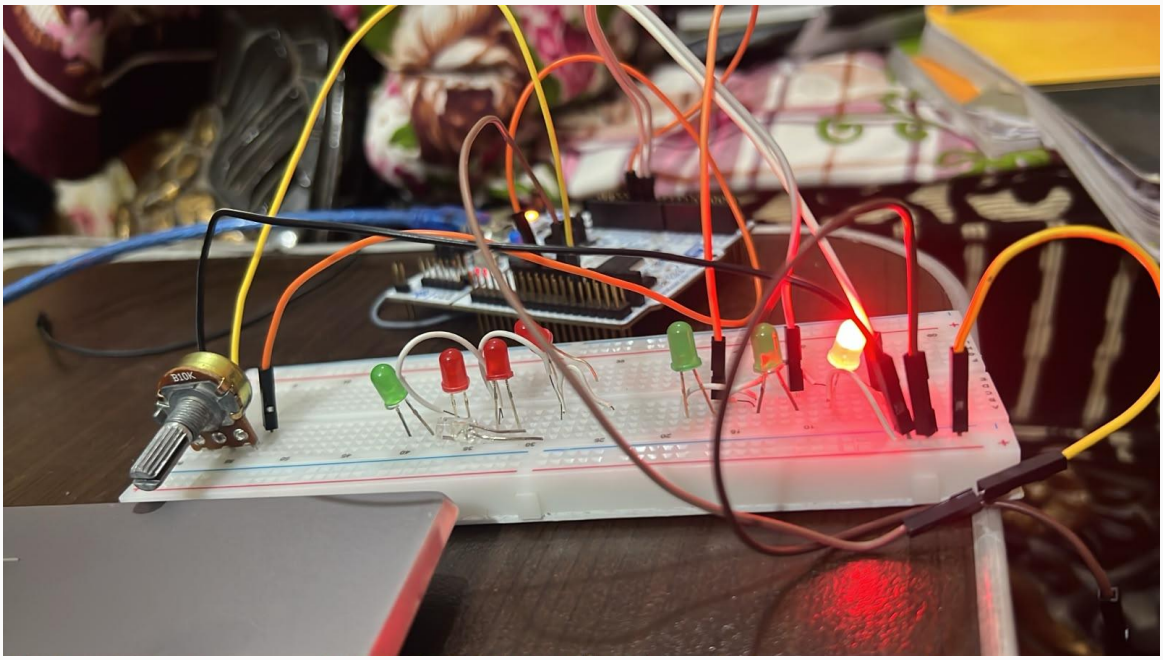
Level 1 Position LOW



Level 2 Position Medium



Level 3 Position High



Result

- The potentiometer successfully provided an analog input to ADC1 Channel 0 (PA0).
- The ADC converted the analog voltage into a 12-bit digital value from 0 to 4095.
- At a low ADC level (0–1364), LED1 (PA5) turned ON.
- At a medium ADC level (1365–2729), LED2 (PA6) turned ON.
- At a high ADC level (2730–4095), LED3 (PA7) turned ON.
- The system successfully demonstrated three-level LED control based on potentiometer position.

Conclusion

The ADC-based LED control system using the STM32 Nucleo-F103RB was successfully implemented. The potentiometer provided an analog signal to PA0, which was converted into a 12-bit digital value using the ADC. Based on the ADC value, one of the three LEDs connected to PA5, PA6, and PA7 was activated. The project successfully demonstrated analog-to-digital conversion and practical ADC-based control.

Github Repositary:

https://github.com/Ahsan-web-hue/STM32-Nucleo-F103RB-ADC-Based-3-Level-LED-Control.git